

# Projektna dokumentacija

*Predmet: Arhitektura softverskih sistema*

*Naziv projekta: Sistem za upravljanje voznim parkom (Fleet Management System)*

**Student:** Milan Ristić

**Broj indeksa:** M522/2025

---

## 1. Opis sistema i analiza zahteva

### 1.1 Opis problema i cilj sistema

U modernom poslovanju, efikasno upravljanje transportnim resursima predstavlja ključni faktor za smanjenje troškova i optimizaciju vremena. Cilj ovog projekta je razvoj **Sistema za upravljanje voznim parkom (Fleet Management System)** koji automatizuje evidenciju vozila, vozača, održavanja (servisa) i raspoređivanja vožnji.

Sistem rešava sledeće uobičajene probleme u praksi:

- Sprečavanje slanja zauzetog ili neispravnog vozila na novu vožnju.
- Sprečavanje angažovanja vozača koji već ima aktivnu vožnju.
- Automatsko vođenje istorije tehničkih pregleda i servisa.
- Automatsko beleženje sistemskih događaja (logova) radi lakše kontrole rada.

### 1.2 Funkcionalni zahtevi

Sistem implementira ukupno 7 Use Case-ova raspoređenih kroz logičke celine (module):

- Registracija vozila (Vehicles):** Unos novog vozila u sistem sa bazičnim podacima (marka, model, registracija, početna kilometraža).
- Registracija vozača (Drivers):** Unos vozača u sistem (ime, prezime, broj vozačke dozvole).
- Pokretanje vožnje (Trips):** Povezivanje slobodnog vozila i slobodnog vozača, upis rute i kreiranje aktivne vožnje.
- Završetak vožnje (Trips - Složena operacija):** Unos krajnje kilometraže vozila, provera ispravnosti podataka, oslobađanje vozila i vozača, ažuriranje kilometraže i okidanje sistemskog događaja TripCompletedEvent.

5. **Evidentiranje servisa (Maintenance):** Slanje slobodnog vozila na servis, promena njegovog statusa u InMaintenance i kreiranje trajnog zapisa o popravci.
6. **Vraćanje vozila sa servisa (Maintenance):** Vraćanje vozila u status Available nakon uspešno završene popravke.
7. **Automatsko kreiranje obaveštenja (Notifications):** Reakcija na događaj završetka vožnje i upisivanje tekstualnog loga u bazu podataka.

### 1.3 Nefunkcionalni zahtevi

- **Održivost (Maintainability):** Izmenom ili dodavanjem novih funkcionalnosti u jednom modulu ne sme se ugroziti stabilnost i rad ostalih modula.
  - **Testabilnost (Testability):** Poslovna logika (Domain i Application sloj) mora biti potpuno izolovana od grafičkog interfejsa (WPF) i eksternih servisa kako bi se omogućilo lako pisanje jediničnih testova.
  - **Labavo povezivanje (Loose Coupling):** Moduli ne smeju direktno zavisiti jedni od drugih. Komunikacija se obavlja preko zajedničkog jezgra (SharedKernel) i mehanizma događaja.
- 

## 2. Arhitektonske odluke i modelovanje

Sistem je projektovan primenom savremenih arhitektonskih šablona koji se koriste u razvoju kompleksnih enterprise sistema:

### 2.1 Modularni monolit

Aplikacija je strukturisana kao **modularni monolit**. Svi funkcionalni moduli su fizički razdvojeni u posebne projekte (Class Libraries) unutar istog rešenja (Solution). Na ovaj način se sprečava nekontrolisano mešanje koda i obezbeđuje jasna granica (boundary) između domenskih oblasti.

### 2.2 Clean i Heksagonalna arhitektura (Ports and Adapters)

Unutar svakog modula primenjena je podela na slojeve prema principima Clean i Heksagonalne arhitekture:

- **Domain sloj:** Nalazi se u centru svakog modula. Sadrži čiste C# entitete i biznis pravila. Svojstva entiteta su zaštićena (private/protected set) kako bi se osigurala enkapsulacija podataka i sprečilo njihovo menjanje mimo domenskih pravila.
- **Application sloj (Ports):** Sadrži Use Case-ove (orkestratore) koji sprovode poslovne zahteve. U ovom sloju se nalaze **Portovi** (interfejsi za spoljne zavisnosti, npr. IVehicleRepository).

- **Infrastructure sloj (Adapters):** Nalazi se na spoljnoj granici modula. Sadrži **Adaptore** – konkretne implementacije portova (npr. `JsonVehicleRepository` koji upisuje podatke na disk).

## 2.3 Dependency Injection (DI) i Composition Root

Sve komponente u sistemu su labavo povezane preko interfejsa. WPF pokretački projekat (`FleetManagement.WPF`) služi kao **Composition Root**. U fajlu `App.xaml.cs` koristi se zvanični `Microsoft.Extensions.DependencyInjection` kontejner za konfiguraciju i registraciju svih repozitorijuma, use case-ova, viewmodel-a i sistemskih servisa.

## 2.4 Event-Driven komunikacija (Event Bus)

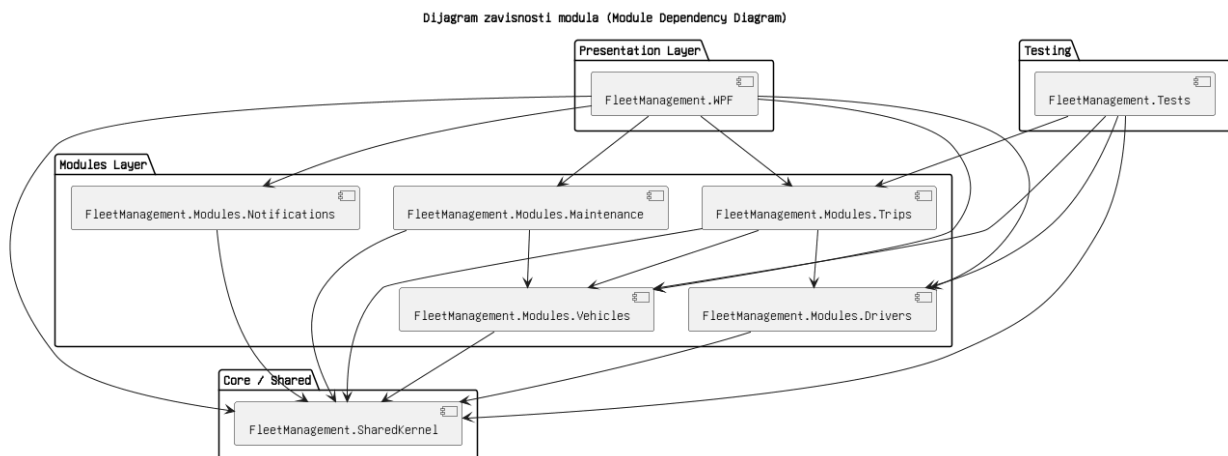
Da bi se izbegla čvrsta zavisnost između `Trips` i `Notifications` modula, implementiran je asinhroni in-process **Event Bus** unutar `SharedKernel`-a. Prilikom završetka vožnje, `Trips` modul samo objavljuje događaj `TripCompletedEvent` na sabirnicu podataka. `Notifications` modul osluškuje ovaj događaj preko `TripCompletedEventHandler`-a i reaguje upisivanjem obaveštenja, bez ikakve direktne zavisnosti od `Trips` modula.

---

## 3. Dijagrami arhitekture

### 3.1 Dijagram zavisnosti modula (Dependency Diagram)

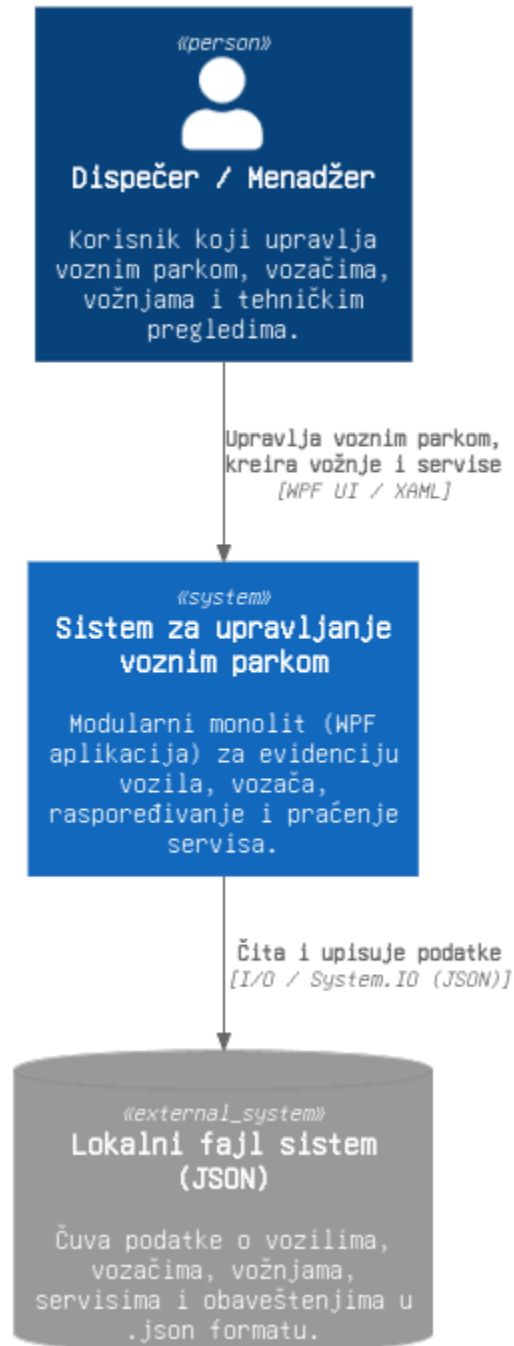
Ovaj dijagram prikazuje fizičke zavisnosti između projekata u Visual Studio rešenju. Primetno je da nema kružnih zavisnosti (ciklusa), a svi funkcionalni moduli zavise isključivo od `SharedKernel`-a.



### 3.2 C4 Model - Nivo 1: Dijagram sistemskog konteksta

Ovaj dijagram prikazuje korisnika (Dispečera/Menadžera), granicu našeg softverskog sistema i lokalni fajl sistem gde se skladište JSON datoteke.

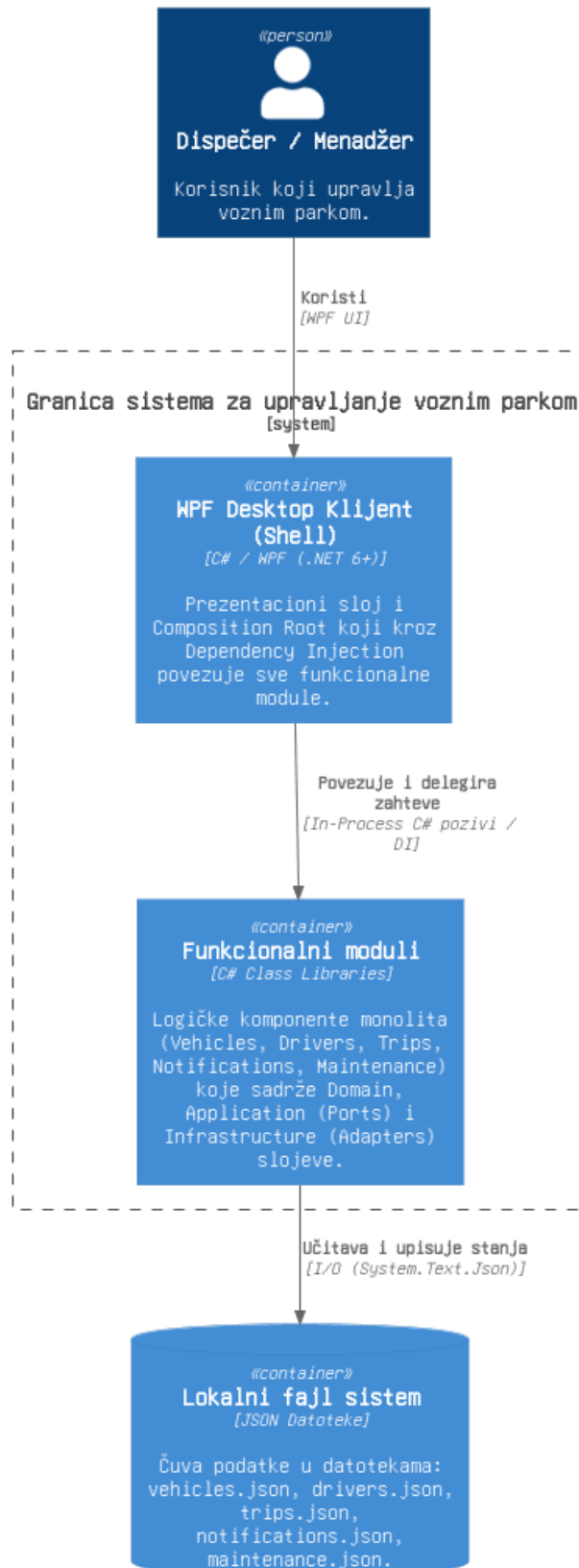
C4 Model - Nivo 1: Dijagram sistemskog konteksta (System Context)



### 3.3 C4 Model - Nivo 2: Dijagram kontejnera

Pošto je naš sistem monolitna desktop aplikacija, on se pokreće kao jedan proces (WPF desktop klijent) koji u sebi učitava logičke module, a podatke zapisuje u lokalne JSON datoteke.

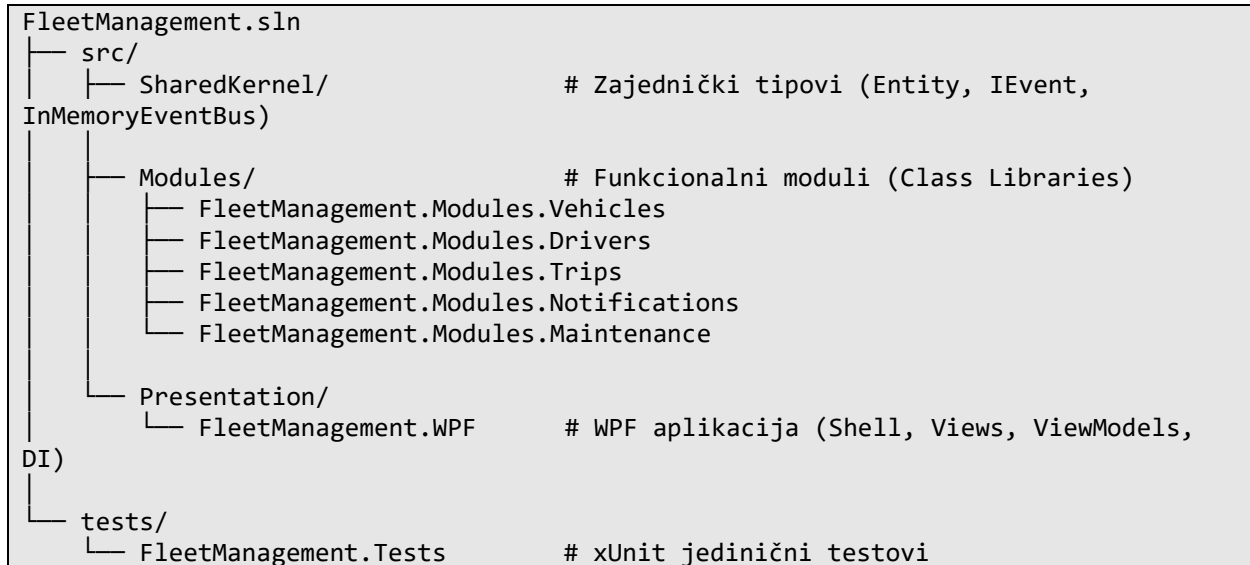
## C4 Model - Nivo 2: Dijagram kontejnera (Container Diagram)



---

## 4. Struktura projekta i slojevi

Visual Studio Solution se sastoji od sledećih fizičkih celina (projekata):



Unutar svakog funkcionalnog modula primenjena je sledeća struktura slojeva:

- Domain/ -> Sadrži domenske entitete (npr. Vehicle.cs, Driver.cs, Trip.cs) i enume.
- Application/ -> Sadrži portove (npr. IVehicleRepository.cs), komande (DTO) i use case-ove (npr. RegisterVehicleUseCase.cs).
- Infrastructure/ -> Sadrži adaptere – konkretne implementacije repozitorijuma (InMemoryVehicleRepository.cs i JsonVehicleRepository.cs).

---

## 5. Kratak opis implementacije i naprednih zahteva

### 5.1 Implementacija poslovne logike

Sva poslovna pravila su striktno enkapsulirana unutar domenskih klasa kako bi se osigurala konzistentnost. Na primer, unutar entiteta Vehicle.cs, metoda AssignToTrip() proverava trenutni status vozila i baca grešku ukoliko vozilo nije slobodno. Time se sprečava da se vozilo koje je na putu ili na servisu dodeli novoj vožnji.

### 5.2 Implementacija naprednih zahteva (CI/CD i Više repozitorijuma)

U cilju postizanja viših ocena, u projektu su uspešno implementirana **tri napredna koncepta**:

1. **CI/CD Pipeline (GitHub Actions):** Kreiran je automatizovani workflow fajl `.github/workflows/dotnet.yml` koji se aktivira pri svakom slanju koda na main ili develop grane. Pipeline na čistom virtuelnom serveru u oblaku preuzima kod, instalira .NET SDK, build-uje rešenje i automatski pokreće svih 5 jediničnih testova.
2. **Više implementacija repository sloja:** Svaki od 5 modula ima dve različite implementacije repozitorijuma – InMemory verziju (koja drži podatke u statičkoj listi) i Json verziju (koja podatke čuva na disku). U datoteci `App.xaml.cs` se jednostavnim menjanjem registracije (npr. `services.AddSingleton<IVehicleRepository, JsonVehicleRepository>()` u `InMemoryVehicleRepository`) menja celokupan način čuvanja podataka u sistemu, bez menjanja ijedne linije koda u domenskom ili aplikacionom sloju.
3. **Mock/Fake implementacije za testiranje:** U projektu `FleetManagement.Tests` napisano je 5 jediničnih testova koji koriste biblioteku **Moq**. Pomoću nje su lažirani svi repozitorijumi, što je omogućilo testiranje use case-ova (npr. `StartTripUseCase`) u potpuno izolovanom okruženju, bez potrebe za upisivanjem fajlova na disk tokom testiranja.

### 5.3 Stabilizacija i nitna bezbednost (Thread Safety)

Tokom integracije sa WPF-om, asinhroni mehanizam za učitavanje podataka sa diska je preusmeren na pozadinsku nit kako se ne bi blokirao grafički interfejs. Da bi se sprečilo pucanje programa usled menjanja `ObservableCollection` kolekcija sa pogrešne niti, u klasi `NotificationsViewModel.cs` i `MaintenanceViewModel.cs` implementiran je bezbedan upis podataka preko **WPF Dispatcher-a** (`Application.Current.Dispatcher.Invoke`).

---

## 6. Zaključak

Primenom modularnog monolita, Clean i Heksagonalne arhitekture razvijen je izuzetno održiv, testabilan i proširiv softverski sistem. Projekat u potpunosti ispunjava sve funkcionalne, tehničke i napredne zahteve propisane specifikacijom ispita. Automatizovani CI/CD proces obezbeđuje dugoročnu stabilnost koda, dok upotreba Dependency Injection mehanizma omogućava laku nadogradnju sistema u budućnosti.